

PRACTICE 3:
STRUCTURAL DESIGN AND CODE
REUTILISATION

Jordi Garcia
Mateu Magem

1. INTRODUCTION

The goal of this practice is to apply and understand structural design by programming a system similar to the one implemented in the second practice.

Structural design is a hierarchical design that consists of dividing our code into smaller entities or modules in order to simplify it and avoid possible errors. In the same manner, we can generalize parameters in order, if necessary, to reuse the code with different requirements on future occasions. Code modularity allows for greater flexibility, simplified debugging and error detection and code reutilisation.

2. PROJECT AND ARCHITECTURE DESCRIPTION

As we have previously stated, the objective of this practice is to program a functionally equal system as the one in the previous practice however by means of structural design.

To accomplish such a purpose, we must first create the necessary projects with their corresponding architecture and entity to subsequently interconnect and integrate them into a master project.

As seen in the second practice, our code consisted of a clock divider that reduced the frequency of the Basys board clock to obtain a reasonable frequency of operation for the 7 segment displays. Another program was implemented with the task of converting a binary number to a BCD encoded number. Finally a 7 segment display decoder was generated in order to show the values in BCD on the 4 displays on the board.

These 3 programs are imported into individual projects.

CLK DIVIDER

```
entity clk_divider is
    generic(eoc: integer:=10000);
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           clk_div : out STD_LOGIC);
end clk_divider;
```

The entity of the divider is the same as the previous practice but with the particularity that we add an integer parameter to generalize the code. This will allow us to change the value of the frequency of the divider at any time without having to modify its architecture.

```
--Divisor de Clock de "eoc" vegades
architecture Behavioral of clk_divider is
    signal div_cnt: integer range 0 to eoc;
begin
    divisor_clk: process (clk, reset) begin
        if clk'event and clk = '1' then
            clk_div <= '0';
            if reset = '1' then
                div_cnt <= 0;
            elsif div_cnt = eoc then
                div_cnt <= 0;
                clk_div <= '1';
            else div_cnt <= div_cnt + 1;
            end if;
        end if;
    end process;
end Behavioral;
```

The only thing that will change in the architecture will be that we will have to set it based on the eoc value. That is, our internal variable *div_cnt* will have to count until it reaches the eoc value.

BIN2BCD

```
entity bin2bcd is
    generic (n: positive:=16);
    Port ( bin_in : in STD_LOGIC_VECTOR (n-1 downto 0);
          clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          bcd0, bcd1, bcd2, bcd3, bcd4: out STD_LOGIC_VECTOR (3 downto 0));
end bin2bcd;
```

This program's entity also contains a generic parameter that corresponding to the extension of the bit vector that read by the conversion function, in our case, with a maximum of 16. Therefore we will have 5 outputs of 4 bits to be able to represent the maximum converted value 2^{16} .

```
architecture Behavioral of bin2bcd is
    signal bcdo: std_logic_vector (15 downto 0) := (others => '0');

    function to_bcd ( bin : std_logic_vector(n-1 downto 0) ) return std_logic_vector is
        variable i : integer:=0;
        variable bcd : std_logic_vector(15 downto 0) := (others => '0');
        variable bint : std_logic_vector(n-1 downto 0) := bin;
    begin
        for i in 0 to n-1 loop -- repeating n-1 times.
            bcd(15 downto 1) := bcd(14 downto 0); --shifting the bits.
            bcd(0) := bint(n-1);
            bint(n-1 downto 1) := bint(n-2 downto 0);
            bint(0) := '0';
            if(i < n-1 and bcd(3 downto 0) > "0100") then --add 3 if BCD digit is greater than 4.
                bcd(3 downto 0) := bcd(3 downto 0) + "0011";
            end if;
            if(i < n-1 and bcd(7 downto 4) > "0100") then --add 3 if BCD digit is greater than 4.
                bcd(7 downto 4) := bcd(7 downto 4) + "0011";
            end if;
            if(i < n-1 and bcd(11 downto 8) > "0100") then --add 3 if BCD digit is greater than 4.
                bcd(11 downto 8) := bcd(11 downto 8) + "0011";
            end if;
            if(i < n-1 and bcd(15 downto 12) > "0100") then --add 3 if BCD digit is greater than 4.
                bcd(15 downto 12) := bcd(15 downto 12) + "0011";
            end if;
        end loop;
        return bcd;
    end to_bcd;

begin
    -- Convertir de binari a BCD
    doubledabble: process(bin_in,bcdo) begin
        bcdo <= to_bcd(bin_in);
        bcd0 <= bcdo (3 downto 0);
        bcd1 <= bcdo (7 downto 4);
        bcd2 <= bcdo (11 downto 8);
        bcd3 <= bcdo (15 downto 12);
        bcd4 <= "0000";
    end process;

end Behavioral;
```

Binary to BCD conversion is implemented with a function that receives a binary-encoded bit vector and returns a BCD-encoded bit vector. The binary vector is moved digit by digit into the BCD vector using a for loop. At each iteration, it is checked that no *nibble* (4-bit vector) of the BCD vector is greater than 0100, and, if it is, 0011 is added to the corresponding nibble. It continues to iterate, until all the digits of the binary vector are shifted to the BCD vector by applying the double dabble algorithm.

Lastly, we relate the value converted to BCD (4 bits) with each of the outputs of our process.

SEG7_CODER

```
entity seg7_coder is
    Port ( char0, char1, char2, char3 : in STD_LOGIC_VECTOR (3 downto 0);
          clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          clk_div : in STD_LOGIC;
          cat : out STD_LOGIC_VECTOR (7 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0));
end seg7_coder;
```

The entity of this program consists of 4 inputs of 4 bits each that correspond to the BCD value of the binary number given by the positions of the switches on the board. We will also have the *clk_div* variable used to activate each of the corresponding anodes at a sufficiently fast frequency to make it simulate that all 4 displays are active at the same time.

```
architecture Behavioral of seg7_coder is
    signal digit_act: integer range 0 to 3;
    signal sg: std_logic_vector (3 downto 0);

begin
    -- 2 Comptador que assigna la entrada que toca
    digit_actual: process(clk, reset) begin
        if clk'event and clk = '1' then
            if reset = '1' then
                digit_act <= 0;
            elsif clk_div = '1' then
                if digit_act = 3 then
                    digit_act <= 0;
                else digit_act <= digit_act + 1;
                end if;
            end if;
        end if;
    end process;
end process;
```

```

        -- Assignar conjunt de switches a sg
    |   case digit_act is
    |       when 0 =>
    |           sg <= char0;
    |       when 1 =>
    |           sg <= char1;
    |       when 2 =>
    |           sg <= char2;
    |       when 3 =>
    |           sg <= char3;
    |   end case;
| end process;

with sg select cat <=
    "00000011" when "0000", --0
    "10011111" when "0001", --1
    "00100101" when "0010", --2
    "00001101" when "0011", --3
    "10011001" when "0100", --4
    "01001001" when "0101", --5
    "01000001" when "0110", --6
    "00011111" when "0111", --7
    "00000001" when "1000", --8
    "00011001" when "1001", --9
    "11111111" when others;

with digit_act select an <=
    "1110" when 0,
    "1101" when 1,
    "1011" when 2,
    "0111" when 3;

| end Behavioral;

```

This architecture consists of reading the number in BCD converted from *bin_in* and showing it on the 4 displays on the board. To do this we use a counter that counts from 0 to 3 at the frequency of *clk_div* in order to read one of the 4 BCD values (group of 4 bits for each BCD value) and send them to the corresponding display (activating the corresponding anode in each case).

MAIN PROGRAM (TOP LEVEL)

```

> entity p31 is
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          sw : in STD_LOGIC_VECTOR (12 downto 0);
          cat : out STD_LOGIC_VECTOR (7 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0));
> end p31;

```

As seen in the image, the external inputs of our system are the board switches, in our case, 13 will be active. We also have the clock input, the reset and the cathodes and anodes corresponding to the outputs. The aim of this program is to relate the previous 3 to this one based on the entities of each one.

```

architecture Behavioral of p31 is

    component clk_divider
        generic (eoc: integer := 100000);
        port ( clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              clk_div : out STD_LOGIC);
    end component;

    component bin2bcd
        generic (n: positive :=16) ;
        port ( bin_in : in STD_LOGIC_VECTOR (n-1 downto 0);
              clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              bcd0, bcd1, bcd2, bcd3, bcd4 : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    component seg7_coder
        port ( char0, char1, char2, char3 : in STD_LOGIC_VECTOR (3 downto 0);
              clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              clk_div : in STD_LOGIC;
              cat : out STD_LOGIC_VECTOR (7 downto 0);
              an : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

```

First of all, we declare the components to be used (3 previous projects) in addition to 5 internal signals that will be used to make connections between components.

```

signal clk_div: STD_LOGIC;
signal bcd0, bcd1, bcd2, bcd3: STD_LOGIC_VECTOR (3 downto 0);

```

Finally, in the body of the architecture, we will connect the inputs and outputs between each of the components, also introducing the necessary generic parameters in each case.

```
begin
u1: clk_divider
    generic map (eoc => 100000)
    port map ( clk => clk,
               reset => reset,
               clk_div => clk_div);

u2: bin2bcd
    generic map (n => 13)
    port map ( bin_in => sw,
               clk => clk,
               reset => reset,
               bcd0 => bcd0,
               bcd1 => bcd1,
               bcd2 => bcd2,
               bcd3 => bcd3,
               bcd4 => open);

u3: seg7_coder
    port map ( char0 => bcd0,
               char1 => bcd1,
               char2 => bcd2,
               char3 => bcd3,
               clk => clk,
               reset => reset,
               clk_div => clk_div,
               cat => cat,
               an => an);

end Behavioral;
```

3. SIMULATION

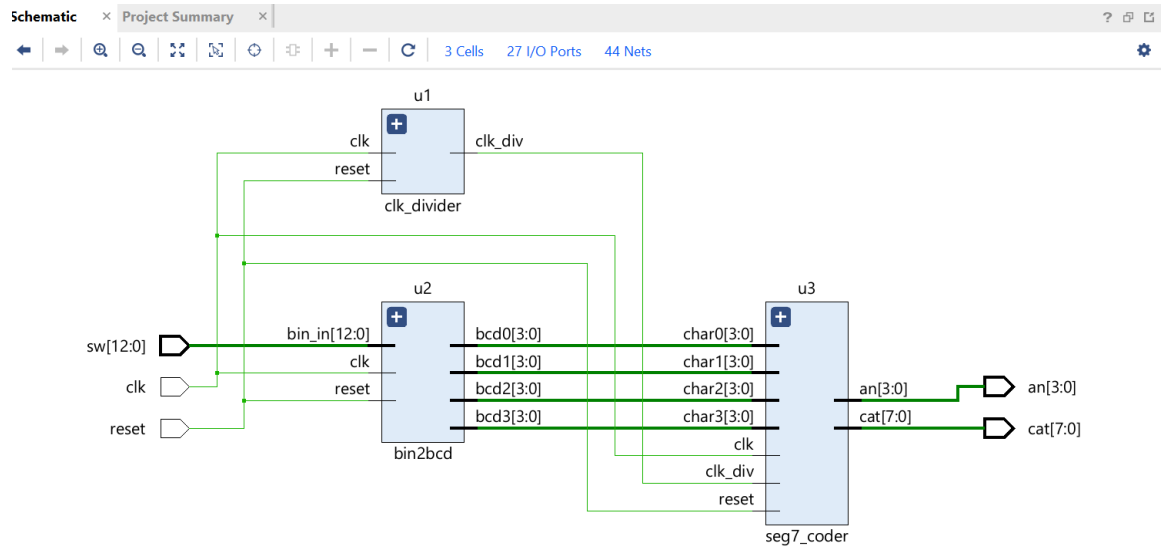
To test the functionality of our system before physically loading it on the board, we simulate it.



As we can see in this small simulation, when a clock and an input from the switches are forced, in this case in hexadecimal, our program will convert it to BCD and at the frequency of *clk_div* it will activate each time a different anode and change the value of the cathodes.

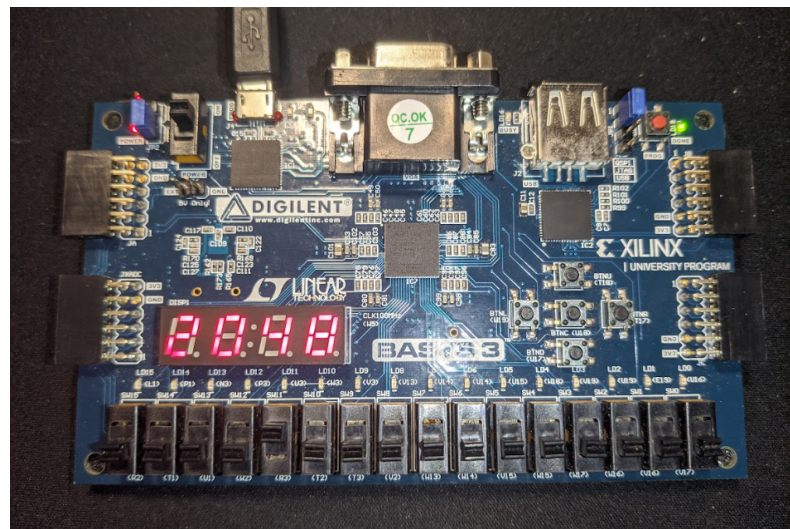
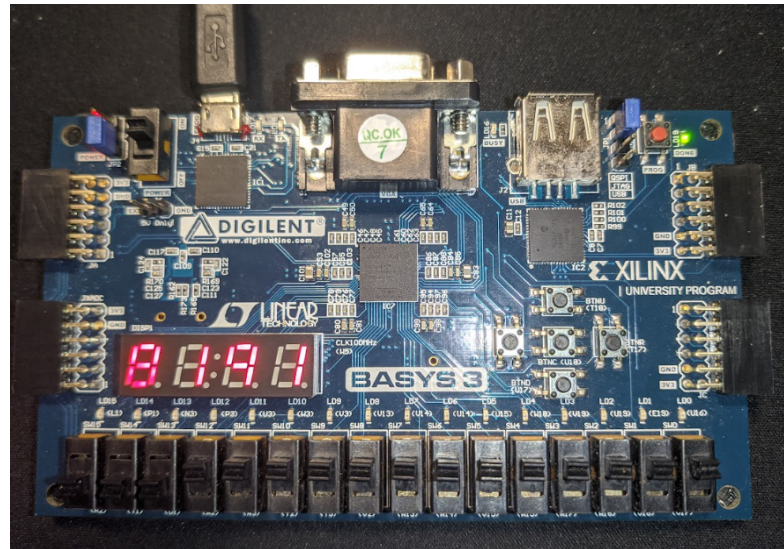
4. RLC ANALYSIS

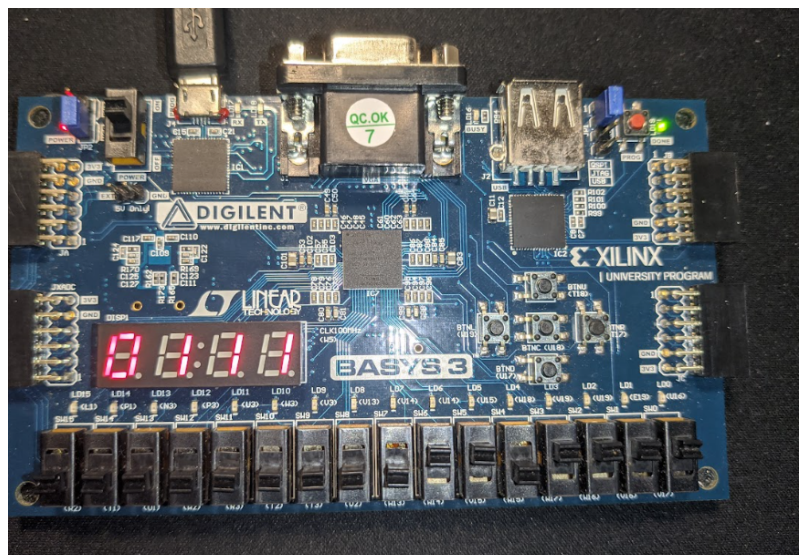
In this section, we can see our design graphically with the connections between ports.



In this same analysis we verify that we have made the correct allocation of I/O pins. Next, we synthesize and implement the program generating the *bitstream*. Finally the program is loaded on the board and the results are visualized.

5. FINAL RESULT





6. BLOCK DESIGN, IP PACKAGING

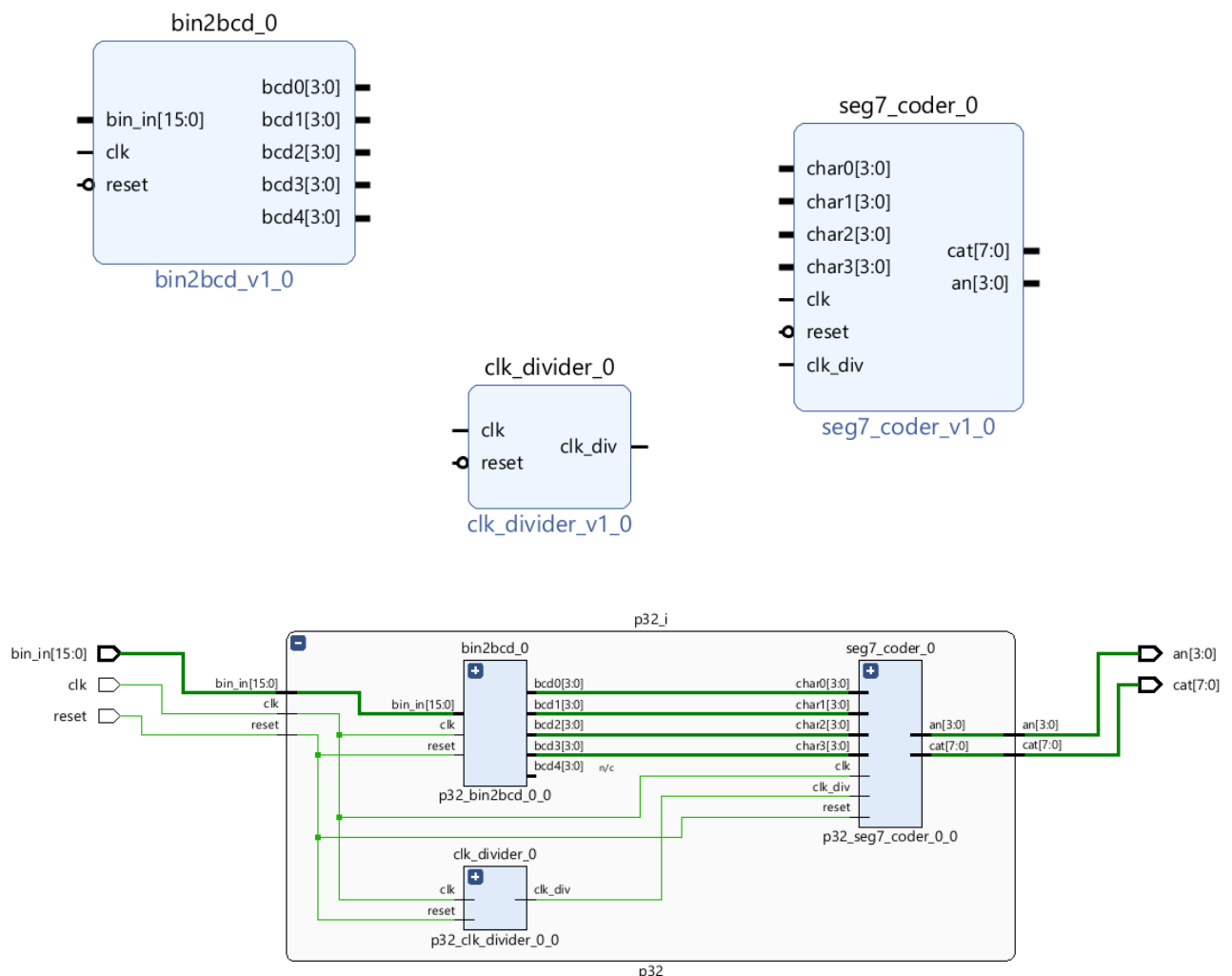
The second part of this practice consists of implementing the same functionality as the first part using but through IP (Intellectual Property) packaging.

In the first part, the used subprograms were:

- clk_divider
- bin2bcd
- seg7_coder

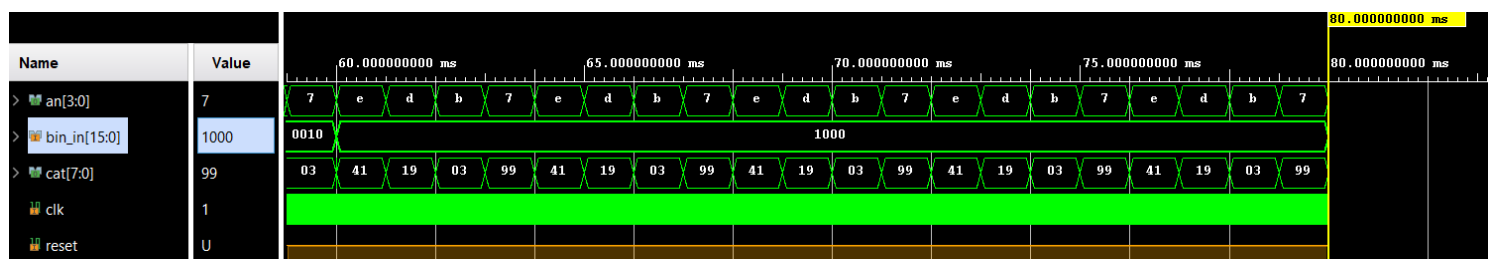
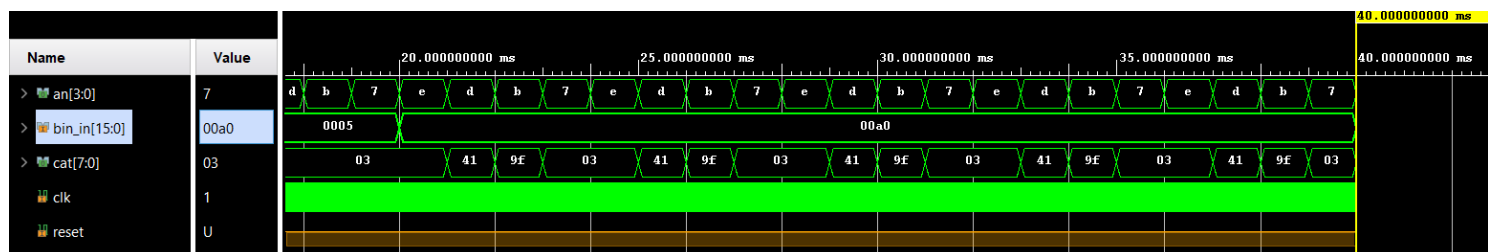
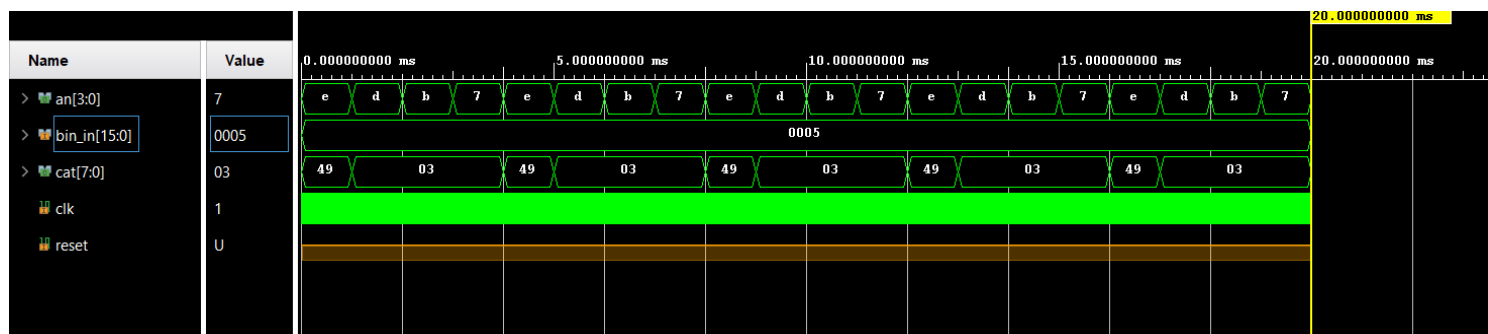
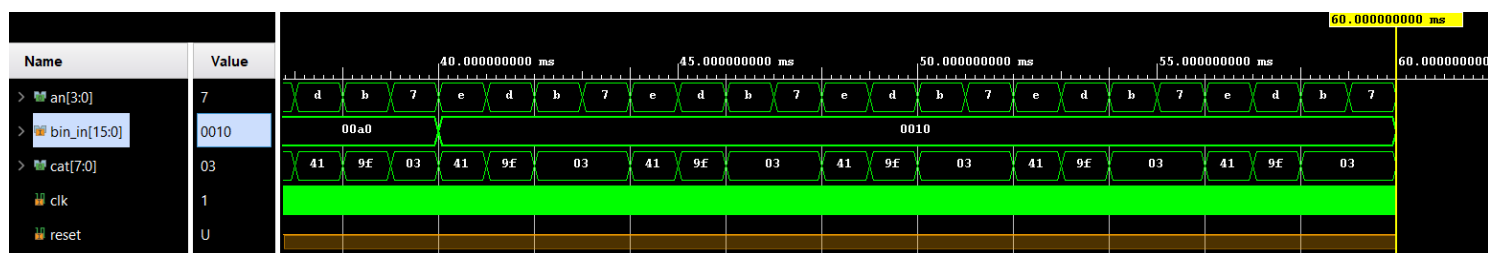
An IP is generated from the projects of each of these subprograms. Then a master project (*top level*) is created combining the three subprograms into one, that is to say, unifying them.

A block diagram is created where the generated IPs are inserted, interconnected with each other and with the input and output ports and the design is validated.

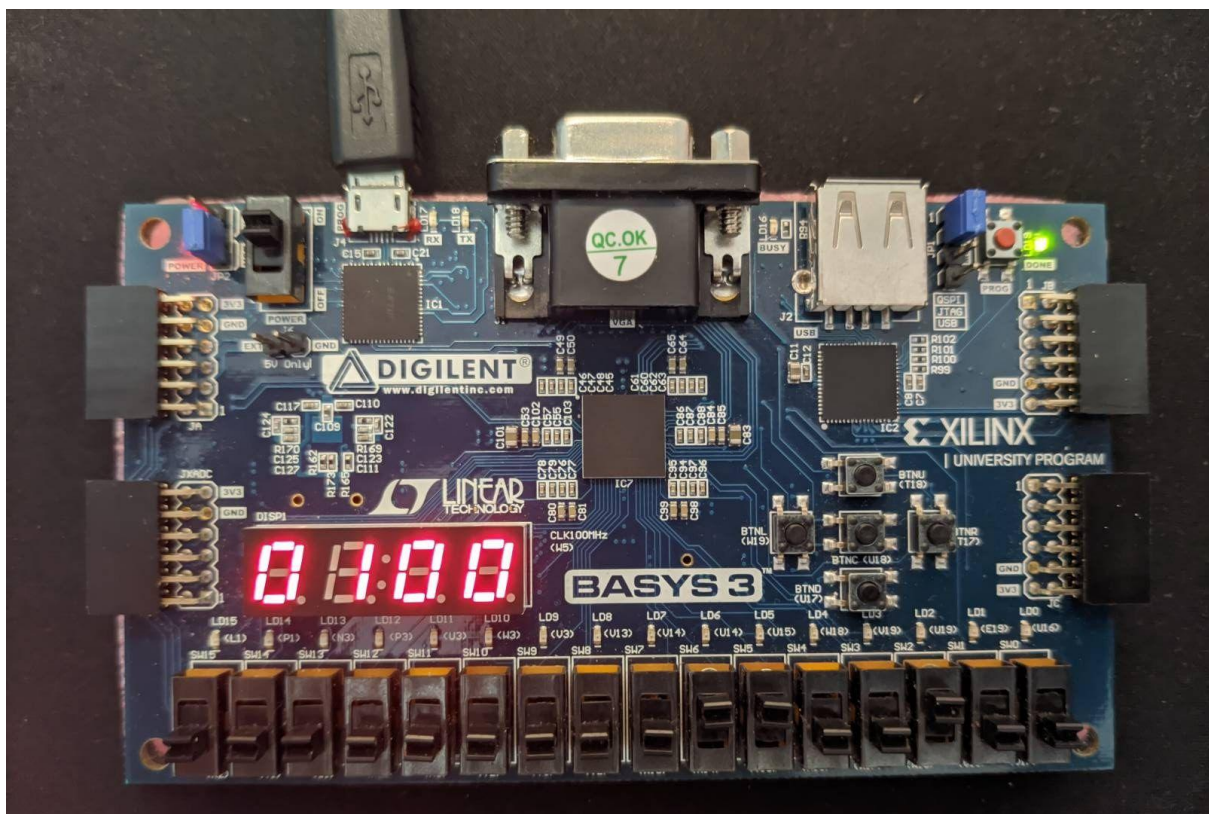
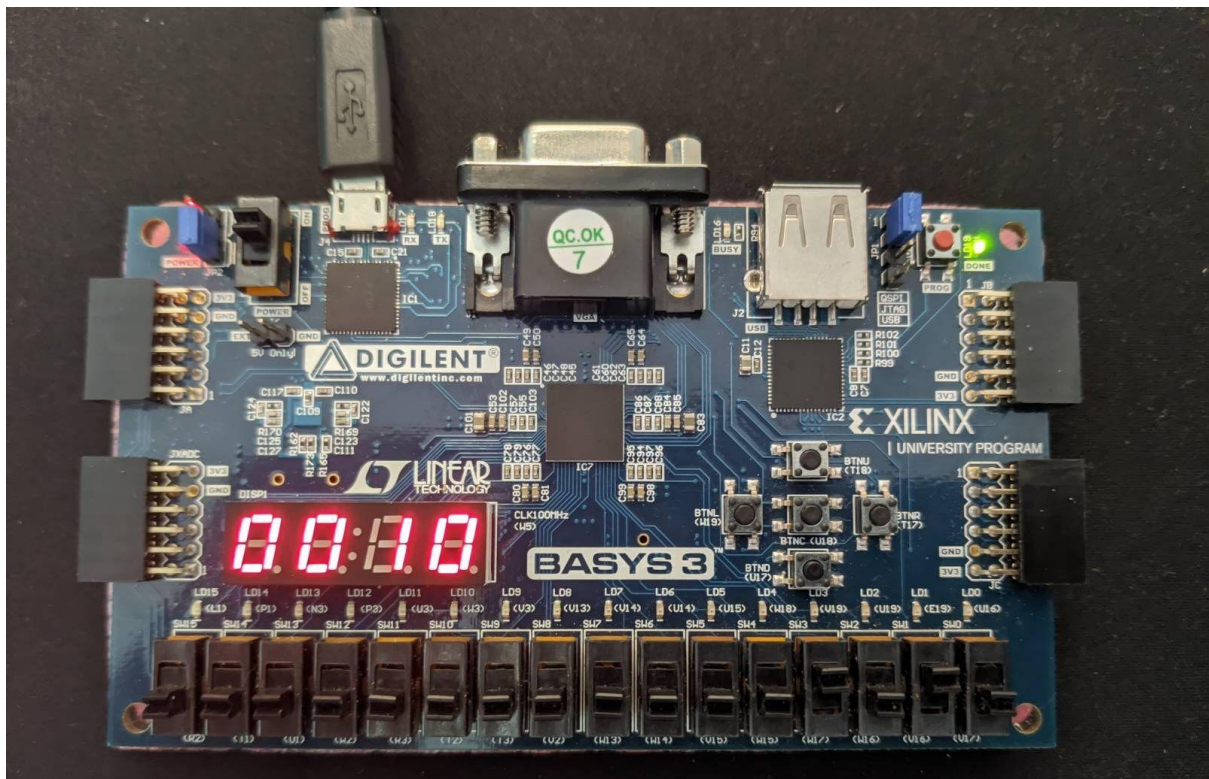


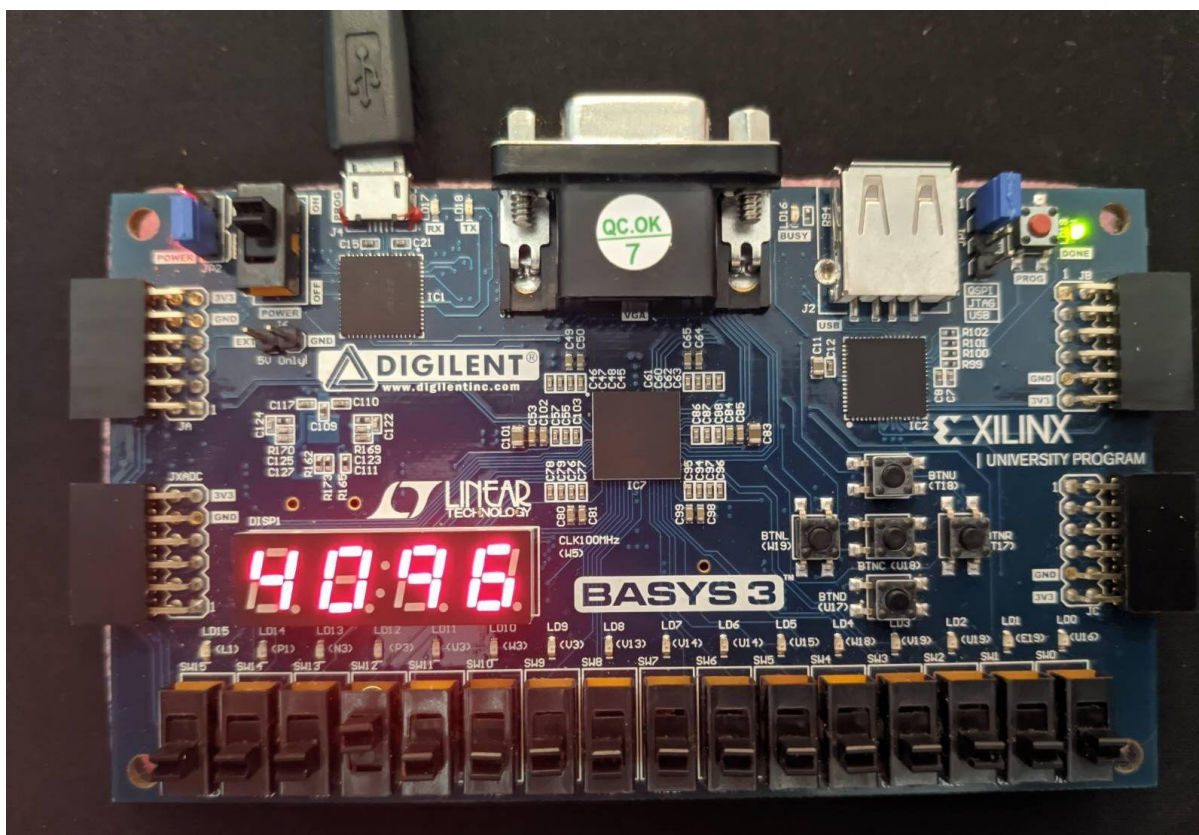
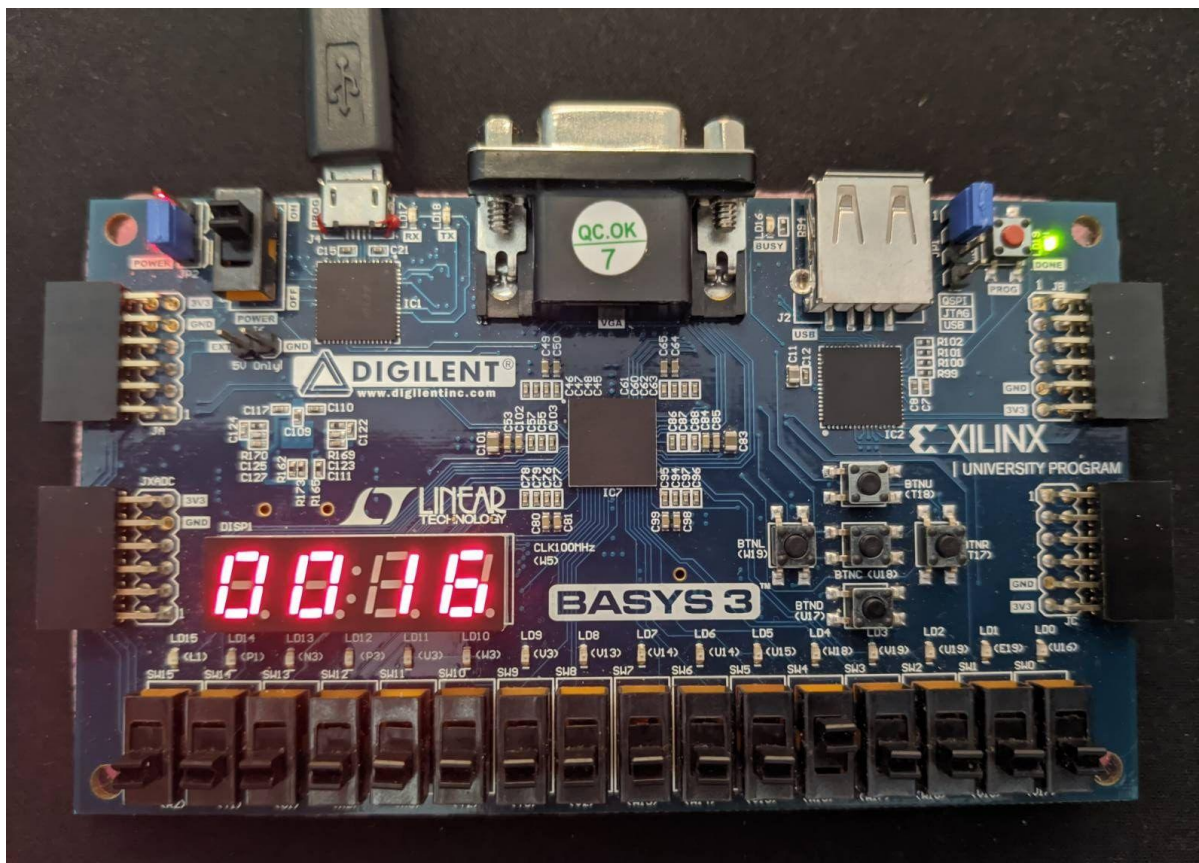
Next, a VHDL Wrapper is generated and the *constraints* file from the previous practice is imported.

The following simulations are carried out to test the correct operation of the program:



In the RTL analysis, the correct assignment of I/O Pins is checked. Finally, it is synthesized, implemented and the bitstream of the program is generated and uploaded to the board to check the final result observed in the following images:





7. CONCLUSIONS

In this practice we have been able to see how the structural/hierarchical design works at a practical level. This type of design helps us structure the system into small entities that ultimately relate to each other. Structural design can be very useful when it comes to avoiding errors or generalizing parameters to make our program more versatile.